

every day. The Web application makes use of that static page to display package information.

The next difficult task with implementing the UI was regarding the option of size calculation around the package widget. Once users select the required packages through check boxes, they can calculate the total size for the custom build. The size must be calculated live.

I used the *memcached* server, which can cache data and program objects. But the size of the object to be cached is restricted to 1 MB. I maintained objects of all repositories with package lists and corresponding package sizes in the *memcached* as cache objects.

Once the user clicks the *Calculate* button, it sends the list of selected packages to a Django function in the backend, which accepts the post requests. From the received list of packages, it analyses the package names one by one, and its size is grabbed from the repository *memcached* object.

The backend handles a lot of complex tasks like dependency resolving, which is very interesting. The user selects a set of packages that are required. But they are not the only packages to be included in the distro build. Each package is dependent on some other package. This relation is called dependency.

Each *pidgin* package (e.g., *pidgin.pidin*) consists of the data and binary files, along with a meta file that describes all the details about the package. This is an XML file. We can find its dependencies by parsing the XML file.

Each package requires some other package to work. For example, Pidgin requires the following packages to work, which in turn may require some other package to work, thus creating a chain of requirements.

Every time a user requests for a size calculation, it resolves dependencies before calculating the size each time and returns a set of information. Here's an extract from the Pidgin package metafile:

```
<Name>pidgin</Name>
<RuntimeDependencies>
<Dependency>audiofile</Dependency>
<Dependency>gtk2</Dependency>
<Dependency>gnutls</Dependency>
<Dependency>gststreamer</Dependency>
<Dependency>startup-notification</Dependency>
<Dependency>cyrus-sasl</Dependency>
<Dependency>gtkspell</Dependency>
<Dependency>avahi-glib</Dependency>
<RuntimeDependencies>
```

The method of solving this complex loop and calculating the list of net packages that should go with the distro build is known as dependency resolving. It is performed in the Pardusman backend. Take a look at the Pardusman code, which you will find interesting.

The wallpaper selection page relies on JQuery. When a user uploads a picture, it is to be stored in the server,



Figure 5: The 'user log' page lets you download the custom distro you created

and only a thumbnail is to be returned and shown in the list of wallpapers. For resizing the uploaded image to a thumbnail, I have used *imagemagick* in the backend.

The wizard involves seven basic steps. At each step, when the next button is clicked, it sends out the data collected from the current page to the server. This data is added to the browser session using the *request.session* dict object available in Django. Once all the wizard steps are completed, it has to generate a configuration file for the custom distro build making use of all the data provided by the user.

XML is used to represent the project configurations. The commonly used XML parser library in Python is ElementTree. But ElementTree is very slow to meet the requirements of this project. Hence, the Pardus team has ported *piksemel*, a XML parser used by the Jabber protocol, to the Pardus platform. It is a very fast XML parser. The *project_config* file is generated with the following tree structure.

```
<PardusmanProject type="install" media="iso">
<Title>Test-Project</Title>
<ReleaseFile>RELEASE.txt</ReleaseFile>
<Wallpaper>user_wallpapers/wallpaper_R0wLAI.jpg</Wallpaper>
<UserContents>user_contents.tar.gz</UserContents>
<PackageSelection repo="Pardus-2009">
<SelectedComponent>x11-server</SelectedComponent>
<SelectedComponent>desktop-fds-base</SelectedComponent>
<SelectedComponent>x11-util</SelectedComponent>
<SelectedPackage>glitz</SelectedPackage>
<SelectedPackage>libdm</SelectedPackage>
<SelectedPackage>ksedu-machle</SelectedPackage>
<Package>low</Package>
<Package>libX11</Package>
<Package>jpeg</Package>
<Package>sysvinit</Package>
<Package>piksemel</Package>
```